

Laços de Repetição em Python

`while` e `for` — Do zero ao prático

🤔 O que é um Laço de Repetição?

Imagine situações do dia a dia:

- 🦷 Escovar os dentes **por 2 minutos** — você repete o movimento várias vezes
- 🏃 Dar **10 voltas** na pista de corrida
- 🔔 Checar o celular **enquanto espera** uma mensagem

*Na programação, laços servem para **executar um bloco de código múltiplas vezes** sem precisar reescrevê-lo.*

O Laço `while`

Executa um bloco **enquanto** uma condição for **verdadeira**.

```
enquanto (condição) → executa o bloco
```

- A condição é verificada **antes** de cada execução
- Quando a condição vira **False**, o laço para
- Ideal quando **não sabemos** quantas vezes iremos repetir

Exemplo Prático — `while`

```
contador = 1

while contador ≤ 5:
    print(contador)
    contador += 1
```

Saída: 1 2 3 4 5

⚠ **Cuidado:** esquecer o `contador += 1` causa um **loop infinito!**

O Laço `for`

Itera sobre uma **sequência definida** de elementos.

```
para (cada item na sequência) → executa o bloco
```

- Sabe exatamente **quantas vezes** vai repetir
- Funciona com listas, strings, `range()` e mais
- Mais **seguro** que `while` para sequências fixas

¹²₃₄ A Função `range()`

Gera uma sequência de números de forma prática:

Sintaxe	Resultado
<code>range(5)</code>	0, 1, 2, 3, 4
<code>range(1, 6)</code>	1, 2, 3, 4, 5
<code>range(0, 10, 2)</code>	0, 2, 4, 6, 8

`range(start, stop, step)` — o **stop** sempre é incluído!

Exemplo Prático — `for`

```
frutas = ["maçã", "banana", "uva"]
```

```
for fruta in frutas:  
    print(fruta)
```

Saída: maçã banana uva

```
for i in range(1, 6):  
    print(i)
```

Saída: 1 2 3 4 5



`while` vs `for`

	<code>while</code>	<code>for</code>
Quando usar	Condição dinâmica	Sequência definida
Nº de repetições	Desconhecido	Conhecido
Risco	Loop infinito	Mais seguro
Exemplo	Aguardar input	Percorrer lista

Controle de Fluxo

break — interrompe o laço imediatamente:

```
for i in range(10):  
    if i == 5:  
        break    # para no 5
```

continue — pula para a próxima iteração:

```
for i in range(5):  
    if i == 2:  
        continue # pula o 2  
    print(i)
```

Conclusão

Resumo:

- `while` → repete enquanto uma condição for verdadeira
- `for` → itera sobre sequências com `range()` ou listas

Laços de repetição são fundamentais em qualquer programa real — domine-os e você resolve a maioria dos problemas de lógica!

Bora praticar! 🐍

Exercícios Práticos

Resolva os exercícios abaixo no **Beecrowd** e consolide o que aprendeu:

#	Problema	Link
1	Intervalo	beecrowd.com — #1072
2	Par ou Ímpar	beecrowd.com — #1074
3	Crescente e Decrescente	beecrowd.com — #1113

💡 Dica: tente resolver sem consultar a solução primeiro. O erro faz parte do aprendizado!

Boa sorte! 🍀